```
        overflow-y: auto;
        margin-bottom: 10px;
        border: 1px solid #ccc;
        padding: 10px;
        border-radius: 5px;
    }
    #chat-box p {
        margin: 0;
        padding: 5px 0;
    }
    #chat-form {
        display: flex;
    }
    #chat-form input {
        flex: 1;
        padding: 10px;
        border: 1px solid #ccc;
        border-radius: 5px 0 0 5px;
    }
    #chat-form button {
        padding: 10px;
        border: 1px solid #ccc;
        border-left: none;
        border-radius: 0 5px 5px 0;
        background-color: #007bff;
        color: white;
        cursor: pointer;
    }
    #chat-form button:hover {
        background-color: #0056b3;
    }
```

5. **Add JavaScript:** Create `static/scripts.js` and add the following JavaScript to handle form submission and updating the chat box:

```
    document.getElementById('chat-form').addEventListener('submit', async function(event) {
        event.preventDefault();
        const messageInput = document.getElementById('message');
        const message = messageInput.value;
        messageInput.value = '';
        const chatBox = document.getElementById('chat-box');
        chatBox.innerHTML += <p><strong>You:</strong> ${message}</p>;
        const response = await fetch('/chat', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/x-www-form-urlencoded'
            },
            body: message=${message}
        });
        const data = await response.json();

        if (data.response) {
            chatBox.innerHTML += <p><strong>AI:</strong> ${data.response}</p>;
        } else {
            chatBox.innerHTML += <p><strong>Error:</strong> ${data.error}</p>;
        }
        chatBox.scrollTop = chatBox.scrollHeight;
    });
```

Testing and Debugging Your Web Application

1. **Run the Flask App:** Start the Flask server by running the following command in your terminal:
   `python app.py`
2. **Access the Web Interface:** Open your web browser and navigate to `http://127.0.0.1:5000`. You should see the chat interface.
3. **Test the Integration:** Enter a message in the input box and click "Send". The AI's response should appear in the chat box.

Enhancing Your ChatGPT Plugin

To make your ChatGPT plugin more robust and feature-rich, consider adding the following enhancements:

1. **User Authentication:** Implement user authentication to personalize the AI experience and manage user sessions.
2. **Conversation History:** Store conversation history in a database to allow users to review past interactions and maintain context.
3. **Custom Commands:** Add custom commands that trigger specific actions, such as setting reminders, checking the weather, or fetching news.
4. **Language Support:** Expand the plugin to support multiple languages by integrating translation APIs or multilingual models.
5. **Error Handling:** Improve error handling to provide more informative feedback to users when something goes wrong.

Developing AI plugins like ChatGPT plugins can significantly enhance the functionality and interactivity of your applications. By following the detailed step-by-step guide in this chapter, you can create, test, and integrate a ChatGPT plugin into your web application. With further enhancements, you can tailor the plugin to meet specific needs and provide a more personalized and engaging user experience.

In the next chapter, we'll explore the integration of AI into smart home environments, enabling you to create intelligent and automated home systems.